

Data Extraction in ETL

Q1. Describe different types of data sources used in ETL with suitable examples.

ETL systems extract data from various sources:

1. **Relational Databases**
 - Examples: MySQL, PostgreSQL, Oracle, SQL Server
 - Data is stored in tables with rows and columns.
 2. **Flat Files**
 - Examples: CSV, TXT files
 - Commonly used for data exchange.
 3. **Spreadsheets**
 - Examples: Microsoft Excel, Google Sheets
 - Frequently used for reporting and small datasets.
 4. **APIs**
 - Examples: Weather API, Twitter API, Payment APIs
 - Provide real-time data access.
 5. **Cloud Storage**
 - Examples: Amazon S3, Google Cloud Storage
 - Stores large volumes of data.
 6. **NoSQL Databases**
 - Examples: MongoDB, Cassandra
 - Used for unstructured and semi-structured data.
-

Q2. What is data extraction? Explain its role in the ETL pipeline.

Data Extraction is the process of collecting data from various source systems before it is transformed and loaded into a target system.

Role in ETL

- First step of ETL (Extract, Transform, Load).
- Collects data from multiple sources.
- Ensures required data is available for processing.
- Provides the foundation for accurate reporting and analytics.

Example:

Extract customer data from MySQL and sales data from CSV files before combining them in a data warehouse.

Data Extraction in ETL

Q3. Explain the difference between CSV and Excel in terms of extraction and ETL usage.

Feature	CSV	Excel
Format	Plain text	Spreadsheet format
Speed	Faster to process	Slower than CSV
File Size	Smaller	Larger
Multiple Sheets	No	Yes
Formatting	Not supported	Supported
ETL Usage	Preferred for large-scale ETL	Used for business reports

Conclusion

CSV files are generally preferred in ETL because they are lightweight and easy to process.

Q4. Explain the steps involved in extracting data from a relational database.

1. Establish a database connection.
2. Authenticate using username and password.
3. Write SQL queries to retrieve required data.
4. Execute queries.
5. Fetch the result set.
6. Validate extracted data.
7. Store data temporarily for transformation.
8. Log extraction activities and errors.

Example:

```
SELECT *  
FROM Customers  
WHERE Country = 'India';
```

Q5. Explain three common challenges faced during data extraction.

Data Extraction in ETL

1. Poor Data Quality

- Missing values
- Duplicate records
- Incorrect formats

2. Large Data Volumes

- Slow extraction process
- Increased memory and storage requirements

3. Source System Availability

- Server downtime
- Network failures
- API rate limits

These issues can delay ETL processes and affect data accuracy.

Q6. What are APIs? Explain how APIs help in real-time data extraction.

API (Application Programming Interface) allows one application to communicate with another.

How APIs Help

- Provide real-time data access.
- Allow automatic data retrieval.
- Support JSON and XML data formats.
- Enable integration with external systems.

Example:

A stock market API provides live stock prices every few seconds.

Benefits:

- Real-time updates
 - Automation
 - Easy integration
-

Data Extraction in ETL

Q7. Why are databases preferred for enterprise-level data extraction?

Databases are preferred because they:

1. Handle large volumes of data efficiently.
2. Provide fast querying capabilities.
3. Support concurrent users.
4. Ensure data integrity and security.
5. Offer backup and recovery features.
6. Scale easily as data grows.

Examples:

- MySQL
- Oracle
- SQL Server
- PostgreSQL

These features make databases suitable for enterprise applications.

Q8. What steps should an ETL developer take when extracting data from large CSV files (1GB+)?

Best Practices

1. Use chunk-based reading instead of loading the entire file.
2. Remove unnecessary columns during extraction.
3. Validate data while reading.
4. Use parallel processing when possible.
5. Compress files to reduce storage and transfer costs.
6. Monitor memory usage.
7. Store intermediate results in staging tables.
8. Maintain detailed logs for errors and progress.

Example (Python)

```
import pandas as pd
```

```
for chunk in pd.read_csv("large_file.csv", chunksize=100000):  
    process(chunk)
```

This approach processes the file in smaller parts and avoids memory issues.

Data Extraction in ETL